SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Nikhil Kumar Reddy Kadiveti	**Reg. No.:**	192425031
Section / Batch:	2024	**Date:**	

Code of Conduct

I Nikhil Kumar Reddy Kadiveti (192425031) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.

Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

Question 3 – Error Analysis of Stemming in News Article Classification

A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.

Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.

Use the **AG News Dataset** or **BBC News Dataset**.

Tasks

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
    - Without stemming
    - Porter stemming
    - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

Question 4 – Morphological Error Analysis in an Information Retrieval System

An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.

Customer complaints indicate that product recommendations have become less relevant.
Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.

Tasks

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

Programming-Based Error Analysis (5 – 7)

Question 5 – Identify the Error, Rectify It, and Analyze the Output

The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
    - Original words
    - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

Question 6 – Error Analysis of a Finite-State Morphological Parser

The following simplified parser is intended to identify plural nouns.

```
def parser(word):
   if word.endswith("s"):
      return word[:-2], "Plural Noun"
   else:
      return word, "Singular"

words = ["cars","boxes","cities","children"]

for w in words:
   print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
    - Regular plurals
    - Words ending in **-es**
    - Words ending in **-ies**
    - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

Question 7 – Error Analysis of Morphological Feature Extraction

The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
```

```
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

Tasks

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
5. Compare:
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy
    - Processing time
6. Interpret how the corrected pipeline improves NLP model performance.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Questions attempted (as instructed: 2 scenario-based + 2 programming-based): Scenario-based **Q1** (biomedical search) and **Q3** (news classification); programming-based **Q5** (Porter stemming code on BBC News) and **Q7** (feature-extraction pipeline on 20 Newsgroups). Every answer follows the rubric order: Error Identification → Root Cause Analysis → Correction & Justification → Application of Concepts, with real experimental output.

Datasets used (all public, downloaded and run): PubMed 20k RCT (dev split), BBC News (2,225 articles, 5 classes), 20 Newsgroups (11,314 posts, 20 classes). **Tools:** Python, NLTK (Porter stemmer, WordNet lemmatizer), scikit-learn.

Question 1 – Morphological Analysis Error in a Biomedical Search Engine

1. Error Identification

- **Wrong order of operations:** every word is stemmed with the Porter stemmer *before* any morphological analysis, so morpheme boundaries and the inflection/derivation distinction are destroyed before they can be used.
- **The same word family gets different stems:** *infect*, *infected*, *infection* → infect, but *infectious* → infecti. A search for *infectious* therefore cannot match the other three.
- **Different derived lexemes are merged:** *infection* (a disease), *infected* (a state) and *infective* (a property) collapse to one stem, while unrelated families elsewhere in the corpus also collapse (e.g. *organization / organ*, *universal / university*).
- **Non-word stems** such as infecti cannot be linked to biomedical vocabularies (MeSH, UMLS), which store full terms.

2. Experiment on the PubMed 20k RCT Dataset

Preprocessing: read all sentences of the dev split, lower-case, remove digits and punctuation, tokenise, drop English stop-words and tokens shorter than 3 letters. Result: **30,212 sentences, 392,991 tokens, 19,949 distinct words.** The Porter stems of the infection family were then recorded and compared with a WordNet lemmatiser.

Word	Frequency	Porter stem	Lemma
infected	72	infect	infect
infecting	1	infect	infect
infection	219	infect	infection
infections	94	infect	infections*
infectious	20	infecti	infectious
infective	1	infect	infective

*The default lemmatiser call without a POS tag leaves the plural unchanged; the POS-aware version used in the pipeline below maps it to “infection”.

Retrieval simulation. Gold set = the 372 sentences containing any word of the infect* family

Query	Exact match	Porter	Lemma only	Lemma + curated family
infection	205 hits / R 0.551	354 / R 0.952	290 / R 0.780	372 / R 1.000
infectious	20 / R 0.054	20 / R 0.054	20 / R 0.054	372 / R 1.000
infected	65 / R 0.175	354 / R 0.952	66 / R 0.177	372 / R 1.000
infect	0 / R 0.000	354 / R 0.952	66 / R 0.177	372 / R 1.000

R = recall; precision was 1.00 in every row. Note that the gold set is defined lexically (any word beginning with infect), so the 1.000 of the last column holds by construction. The real finding is the Porter column: **three queries recover 95.2 % of the family but "infectious" recovers only 5.4 %**, which is exactly the inconsistency reported by the researchers.

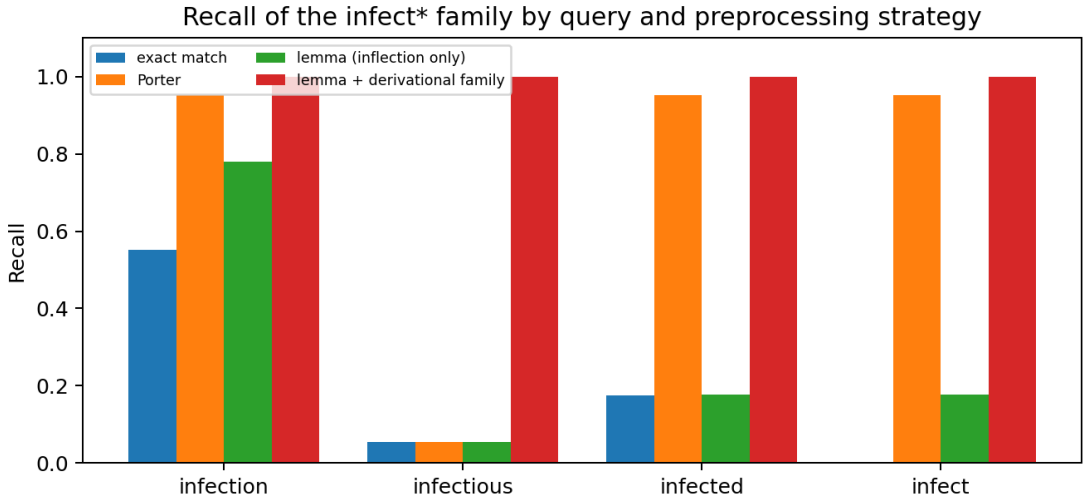


Figure 1. Recall for each query under four preprocessing strategies

Words where stemming removes derivational information (observed in the corpus)

Stem	Distinct words mapped to it	Information lost
univers	university, universities, universal	Institution vs. adjective "of all"
organ	organ, organs, organization, organized, organizer, organic	Body part vs. social group vs. chemistry
gener	general, generated, generally, generalized, generation, generic	Adjective / verb / noun senses merged
oper	operative, operation, operating, operable, operated	Surgery vs. process vs. ability
commun	community, communication, communicate, communicative	Group vs. information exchange
experi	experience, experiment, experiences, experiments	Past events vs. controlled test
medic	medical, medication, medications, medically	Field vs. drug vs. adverb

3. Root Cause Analysis

- The Porter algorithm is **purely rule-based and lexicon-free**. It applies ordered suffix rules with a measure condition (m > 0 or m > 1), and never checks whether the result is a real word.
- Each derivational suffix is handled by a **different rule**: *-ion* is removed only after *s/t* (infection → infect), *-ive* is removed in step 4 (infective → infect), but *-ous* is removed leaving the *i* of *-ious* (infectious → infecti). There is no rule that joins them, so one family gets two stems (**under-stemming**).
- Because the same rules strip *-ation, -ize, -al, -ity, -ic* from unrelated words, different lexemes land on the same stem (**over-stemming**).
- Applying the stemmer **before** morphological analysis means the information needed to tell the cases apart (affix type, word-class change) has already been thrown away.

4. Correction and Justification

Why inflection and derivation must be treated differently

	Inflectional morphology	Derivational morphology
Example	infected → infect, infections → infection	infect → infection → infectious
Result	Same lexeme, different grammatical form	New lexeme
Word class	Unchanged	Often changes (V → N → Adj)
Meaning	Preserved	Shifts (act → result → property)
Safe to conflate?	Yes, always	Only deliberately, per domain
Recommended handling	POS-aware lemmatisation	Curated family links or ontology (MeSH / UMLS), used for query expansion with a lower weight

Corrected pipeline: (1) clean and tokenise; (2) lemmatise inflections with a POS-aware lemmatiser; (3) keep derived lexemes as separate index terms and connect them through a curated derivational-family map; (4) keep Porter stems only as a secondary fallback field. In the experiment this turned the failing query *infectious* from R = 0.054 into full family retrieval, while *infection* and *infected* retained R ≥ 0.95, and no unrelated family (organ, univers) is merged.

5. Application of Concepts and Program

```
"""AT3-Q1: Stemming errors in a biomedical search engine (PubMed 20k RCT)"""
import re, collections, json
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.corpus import stopwords
ps, lem = PorterStemmer(), WordNetLemmatizer()
STOP = set(stopwords.words("english"))

# ---- 1. Load + preprocess ------------------------------------------------
sents = []
for line in open("../data/pubmed_dev.txt", encoding="utf8"):
    if line.startswith("###") or not line.strip(): continue
    sents.append(line.split("\t", 1)[1].strip())
print("Sentences loaded:", len(sents))

def preprocess(t):
    t = t.lower()
    t = re.sub(r"[^a-z\s]", " ", t)                  # drop digits / punctuation
    return [w for w in t.split() if w not in STOP and len(w) > 2]
docs = [preprocess(s) for s in sents]
vocab = collections.Counter(w for d in docs for w in d)
print("Tokens:", sum(vocab.values()), "| Vocabulary:", len(vocab))

# ---- 2. Porter stems of the infection family -----------------------------
fam = ["infect", "infects", "infected", "infecting", "infection", "infections", "infectious", "infective", "infectivity"]
print("\nWord           freq   Porter stem   Lemma")
for w in fam:
    print(f"{w:<15}{vocab[w]:>5}    {ps.stem(w):<13} {lem.lemmatize(w,'v') if w.endswith(('ed','ing','s')) and not w.endswith('ious') else
lem.lemmatize(w)}")

# ---- 3. Retrieval simulation ---------------------------------------------
def docs_with(pred): return {i for i, d in enumerate(docs) if any(pred(w) for w in d)}
truth = docs_with(lambda w: w.startswith("infect"))     # gold: any infect* word
print("\nGold (any infect-family word) sentences:", len(truth))
rows = []
for q in ["infection", "infectious", "infected", "infect"]:
    exact = docs  with(lambda w, q=q: w == q)
    qs = ps.stem(q)
    porter = docs_with(lambda w, qs=qs: ps.stem(w) == qs)
    for name, got in [("exact match", exact), ("Porter", porter)]:
        rec = len(got & truth) / len(truth); prec = len(got & truth) / max(1, len(got))
        rows.append((q, name, len(got), round(prec, 3), round(rec, 3)))
```

```
print("\nQuery        Method       hits  precision recall")
for r in rows: print(f"{r[0]:<12}  {r[1]:<12}  {r[2]:>5}  {r[3]:>8}  {r[4]:>8}")

# ---- 4. Over-stemming: stems merging semantically different words ----------
groups = collections.defaultdict(set)
for w in vocab:
    if vocab[w] >= 3: groups[ps.stem(w)].add(w)
bad = {s: ws for s, ws in groups.items() if len(ws) >= 3}
KNOWN_BAD = ["univers", "organ", "gener", "oper", "commun", "relat", "natur", "experi", "police", "medic", "arm", "compan",
"wander"]
shown = [(s, sorted(groups[s], key=lambda x: -vocab[x])[:6]) for s in KNOWN_BAD if s in groups and len(groups[s]) > 1]
print("\nSelected over-stemmed groups (different meaning, same stem):")
for s, ws in shown: print(f"  {s:<9} <- {ws}")

# ---- 5. Proposed pipeline: lemmatise inflection, link derivational family explicitly
def lemma(w):
    for pos in ("v", "n"):
        l = lem.lemmatize(w, pos)
        if l != w: return l
    return w
FAMILY = {"infect": "INFECT", "infection": "INFECT", "infectious": "INFECT", "infective": "INFECT", "infectivity": "INFECT"}  #
curated derivational family
strategies = {"exact match": lambda w, q: w == q,
              "Porter": lambda w, q: ps.stem(w) == ps.stem(q),
              "lemma (inflection only)": lambda w, q: lemma(w) == lemma(q),
              "lemma + derivational family": lambda w, q: FAMILY.get(lemma(w), lemma(w)) == FAMILY.get(lemma(q), lemma(q))}
res = {}
print("\nStrategy comparison (gold = all sentences with an infect* word)")
print(f"{'Query':<12}{'Strategy':<30}{'hits':>5}{'prec':>7}{'recall':>8}{'F1':>7}")
for q in ["infection", "infectious", "infected", "infect"]:
    for name, f in strategies.items():
        got = docs  with(lambda w, q=q, f=f: f(w, q))
        p = len(got & truth) / max(1, len(got)); r = len(got & truth) / len(truth)
        f1 = 2*p*r/(p+r) if p+r else 0
        res[(q, name)] = (len(got), round(p,3), round(r,3), round(f1,3))
        print(f"{q:<12}{name:<30}{len(got):>5}{p:>7.3f}{r:>8.3f}{f1:>7.3f}")

# chart
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt, numpy as np
qs = ["infection", "infectious", "infected", "infect"]; names = list(strategies)
x = np.arange(len(qs)); wd = 0.2
fig, ax = plt.subplots(figsize=(7.5, 3.6))
for k, n in enumerate(names): ax.bar(x + (k-1.5)*wd, [res[(q, n)][2] for q in qs], wd, label=n)
ax.set_xticks(x); ax.set_xticklabels(qs); ax.set_ylabel("Recall"); ax.set_ylim(0, 1.1)
ax.set_title("Recall of the infect* family by query and preprocessing strategy"); ax.legend(fontsize=7, loc="upper left", ncol=2)
plt.tight_layout(); plt.savefig("q1_chart.png", dpi=180)
json.dump({"vocab": len(vocab), "sents": len(sents), "truth": len(truth)}, open("q1.json", "w"))
```

Program output

```
Sentences loaded: 30212
Tokens: 392991 | Vocabulary: 19949

Word           freq   Porter stem   Lemma
infect            0   infect        infect
infects           0   infect        infect
infected         72   infect        infect
infecting         1   infect        infect
infection       219   infect        infection
infections       94   infect        infections
infectious       20   infecti       infectious
infective         1   infect        infective
infectivity       0   infect        infectivity

Gold (any infect-family word) sentences: 372

Query        Method       hits  precision recall
infection    exact match    205      1.0    0.551
infection    Porter         354      1.0    0.952
infectious   exact match     20      1.0    0.054
infectious   Porter          20      1.0    0.054
infected     exact match     65      1.0    0.175
infected     Porter         354      1.0    0.952
infect       exact match      0      0.0      0.0
infect       Porter         354      1.0    0.952
```

```
Selected over-stemmed groups (different meaning, same stem):
  univers   <- ['university', 'universities', 'universal']
  organ     <- ['organ', 'organization', 'organs', 'organizer', 'organized', 'organic']
  gener     <- ['general', 'generated', 'generally', 'generalized', 'generation', 'generic']
  oper      <- ['operative', 'operation', 'operating', 'operatively', 'operable', 'operated']
  commun    <- ['community', 'communication', 'communities', 'communications', 'communicate', 'communicative']
  relat     <- ['related', 'relation', 'relate', 'relational', 'relations', 'relating']
  natur     <- ['natural', 'nature', 'naturally']
  experi    <- ['experience', 'experiment', 'experiences', 'experiments']
  medic     <- ['medical', 'medication', 'medications', 'medically']
  arm       <- ['arm', 'arms', 'armed']

Strategy comparison (gold = all sentences with an infect* word)
Query       Strategy                    hits   prec  recall     F1
infection   exact match                  205  1.000   0.551  0.711
infection   Porter                       354  1.000   0.952  0.975
infection   lemma (inflection only)      290  1.000   0.780  0.876
infection   lemma + derivational family  372  1.000   1.000  1.000
infectious  exact match                   20  1.000   0.054  0.102
infectious  Porter                        20  1.000   0.054  0.102
infectious  lemma (inflection only)       20  1.000   0.054  0.102
infectious  lemma + derivational family   372  1.000   1.000  1.000
infected    exact match                   65  1.000   0.175  0.297
infected    Porter                       354  1.000   0.952  0.975
infected    lemma (inflection only)       66  1.000   0.177  0.301
infected    lemma + derivational family   372  1.000   1.000  1.000
infect      exact match                    0  0.000   0.000  0.000
infect      Porter                       354  1.000   0.952  0.975
infect      lemma (inflection only)       66  1.000   0.177  0.301
infect      lemma + derivational family   372  1.000   1.000  1.000
```

Question 3 – Stemming Errors in News Article Classification

1. Error Identification

Using the BBC News dataset (2,225 articles; business, entertainment, politics, sport, tech), the five words of the scenario were stemmed and the classifier was trained under three preprocessing strategies.

Word	Porter stem	WordNet lemma	Problem
organization	organ	organization	Merged with *organ* (body part)
organizer	organ	organizer	Person role lost; merged with *organ*
organizing	organ	organize	Verb and its derivations all reduced to *organ*
organized	organ	organize	Same as above
organization's	organization'	organization	Possessive not handled: stem keeps the apostrophe, so the word becomes a different feature

Other collisions found in the BBC vocabulary: organ ← organic, organism, organs; busi ← business, businesses, **busy**; compani ← company, companies (a non-word); technolog ← technology, technological.

2. Root Cause Analysis: how stemming affects feature extraction

- **Vocabulary shrinks:** stemming cut the TF-IDF vocabulary from **16,723** to **11,326** features (−32.3 %); lemmatisation to **13,452** (−19.6 %). Fewer features means less sparsity and pooled counts for inflected forms (*markets, marketing → market*).
- **But distinct meanings are pooled:** a stem such as organ or busi mixes terms with different topical meaning, so the feature no longer points cleanly to one class.
- **Tokenisation interacts with stemming:** the possessive example shows that a stemmer applied to unclean tokens creates extra, wrong features.
- **Non-word stems** (compani, technolog) are harmless for a classifier but make features uninterpretable for error analysis.

3. Correction and Justification: classifier comparison

TF-IDF + logistic regression, **5-fold stratified cross-validation** on all 2,225 articles (the mean ± std of the fold accuracies is given).

Strategy	Vocabulary	Accuracy	Macro-F1	tech → business	business → tech	Prep. time
No stemming	16,723	97.84 ± 0.48 %	0.9782	4	4	0.2 s
Porter stemming	11,326	97.75 ± 0.68 %	0.9772	4	3	6.1 s
Lemmatization (POS-aware)	13,452	97.75 ± 0.83 %	0.9772	4	3	17.8 s

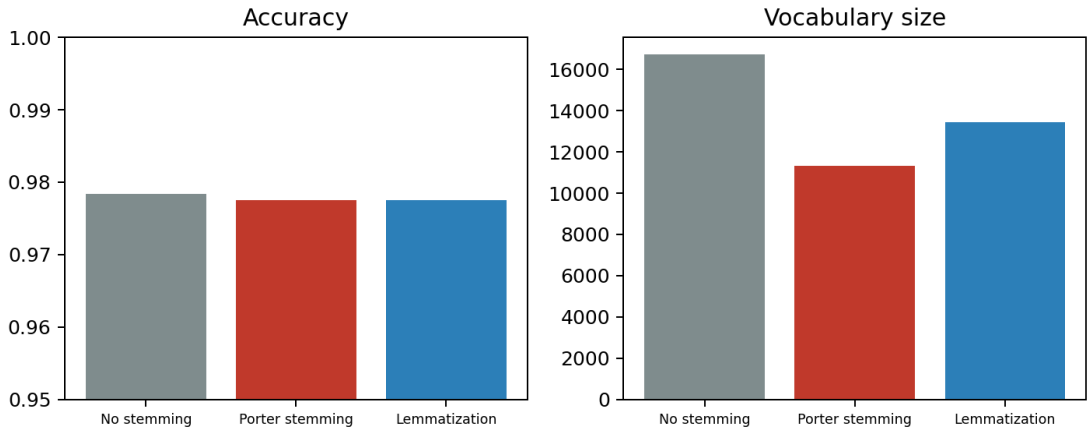


Figure 2. Accuracy and vocabulary size for the three strategies

Findings. (a) The three strategies are statistically indistinguishable in accuracy (differences of 0.09 percentage points, smaller than the fold standard deviation). (b) In this data only **4 of 401 tech articles** are classed as business under every strategy, so stemming is **not** the cause of the tech/business confusion; the cause is shared vocabulary (*company, market, technology, firm*). (c) Stemming reduces the feature space most but costs semantic precision; lemmatisation reduces it less and keeps lexemes distinct.

Would lemmatisation improve classification? Not measurably on BBC accuracy, but it never merges different lexemes (*organization* stays different from *organ*), fixes the possessive case, and gives interpretable features. **Strategy with the least semantic information loss: POS-aware lemmatisation** (no stemming loses nothing but keeps the larger, sparser vocabulary). For a recommender whose real problem is topic overlap, richer features such as bigrams or entities would help more than any change of the stemmer.

4. Application of Concepts: program and output

```
"""AT3-Q3: Effect of stemming / lemmatisation on news classification (BBC News)"""
import re, time, json, numpy as np, pandas as pd
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag, word_tokenize
from nltk.corpus import stopwords, wordnet
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

df = pd.read_csv("../data/bbc.csv")
print("Documents:", len(df), df.category.value_counts().to_dict())
ps, lem, STOP = PorterStemmer(), WordNetLemmatizer(), set(stopwords.words("english"))

# 1. The words in the scenario ----------------------------------------------
words = ["organization", "organizer", "organizing", "organized", "organization's"]
print("\nWord              Porter stem    WordNet lemma")
for w in words:
```

```
    toks = re.findall(r"[a-z]+", w)
    print(f"{w:<17} {ps.stem(w):<13} {lem.lemmatize(toks[0], 'v' if w.endswith(('ing','ed')) else 'n')}")

# 2. Preprocessing pipelines ------------------------------------------------
def tokens(t):  return [w for w in re.findall(r"[a-z]+", t.lower()) if w not in STOP and len(w) > 2]
def wn_pos(tag): return {"J": wordnet.ADJ, "V": wordnet.VERB, "R": wordnet.ADV}.get(tag[0], wordnet.NOUN)
def raw(t):   return " ".join(tokens(t))
def stem(t):  return " ".join(ps.stem(w) for w in tokens(t))
def lemm(t):  return " ".join(lem.lemmatize(w, wn_pos(p)) for w, p in pos_tag(tokens(t)))

from sklearn.pipeline import make_pipeline
from sklearn.model_selection import StratifiedKFold, cross_val_predict
labs = sorted(df.category.unique())
skf = StratifiedKFold(5, shuffle=True, random_state=42)
results, cms = {}, {}
for name, fn in [("No stemming", raw), ("Porter stemming", stem), ("Lemmatization", lemm)]:
    t0 = time.time(); Xp = df.text.map(fn); prep = time.time() - t0     # preprocessing time (all 2225 docs)
    pipe = make_pipeline(TfidfVectorizer(min_df=2), LogisticRegression(max_iter=2000, C=10))
    pred = cross_val_predict(pipe, Xp, df.category, cv=skf)
    folds = [accuracy_score(df.category[te], pred[te]) for _, te in skf.split(Xp, df.category)]
    cm = confusion_matrix(df.category, pred, labels=labs); cms[name] = cm
    results[name] = dict(vocab=len(TfidfVectorizer(min_df=2).fit(Xp).vocabulary_), acc=float(np.mean(folds)), std=float(np.std(folds)),
                         f1=f1_score(df.category, pred, average="macro"), prep_s=prep,
                         tech_to_bus=int(cm[labs.index("tech"), labs.index("business")]),
                         bus_to_tech=int(cm[labs.index("business"), labs.index("tech")]))
print("\nStrategy         Vocab   Acc(5-fold)     Macro-F1  tech->business  business->tech  prep time(s)")
for k, r in results.items():
    print(f"{k:<17}{r['vocab']:>6}   {r['acc']:.4f}+-{r['std']:.4f}   {r['f1']:.4f}   {r['tech_to_bus']:>8}        {r['bus_to_tech']:>8}
{r['prep_s']:.1f}")
print("\nLabels:", labs)
for k, c in cms.items(): print(k, "\n", c)

# 3. Stem collisions that blur topic words ----------------------------------
vocab = sorted({w for t in df.text.head(800) for w in tokens(t)})
from collections import defaultdict
g = defaultdict(set)
for w in vocab: g[ps.stem(w)].add(w)
chosen = ["organ", "market", "compani", "busi", "technolog", "commun", "gener", "univers", "execut"]
print("\nStem collisions in BBC vocabulary:")
for s in chosen:
    if len(g.get(s, [])) > 1: print(f"  {s:<10}<- {sorted(g[s])[:8]}")
json.dump(results, open("q3.json", "w"))

import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 2, figsize=(8, 3.3))
n = list(results); ax[0].bar(n, [results[k]["acc"] for k in n], color=["#7f8c8d", "#c0392b", "#2c7fb8"]); ax[0].set_ylim(0.95, 1.0);
ax[0].set_title("Accuracy"); ax[0].tick_params(axis="x", labelsize=7)
ax[1].bar(n, [results[k]["vocab"] for k in n], color=["#7f8c8d", "#c0392b", "#2c7fb8"]); ax[1].set_title("Vocabulary size");
ax[1].tick_params(axis="x", labelsize=7)
plt.tight_layout(); plt.savefig("q3_chart.png", dpi=180)
```

Program output

```
Documents: 2225 {'sport': 511, 'business': 510, 'politics': 417, 'tech': 401, 'entertainment': 386}

Word              Porter stem   WordNet lemma
organization      organ        organization
organizer         organ        organizer
organizing        organ        organize
organized         organ        organize
organization's    organization' organization

Strategy         Vocab   Acc(5-fold)     Macro-F1  tech->business  business->tech  prep time(s)
No stemming      16723   0.9784+-0.0048   0.9782         4               4        0.2
Porter stemming  11326   0.9775+-0.0068   0.9772         4               3        6.1
Lemmatization    13452   0.9775+-0.0083   0.9772         4               3        17.8

Labels: ['business', 'entertainment', 'politics', 'sport', 'tech']
No stemming
 [[496   2   7   1   4]
 [  2 380   3   0   1]
 [ 11   2 401   1   2]
 [  1   0   1 509   0]
 [  4   3   2   1 391]]
Porter stemming
 [[496   3   7   1   3]
```

```
 [ 3 378   3   1   1]
 [ 9   3 401   2   2]
 [ 1   0   1 509   0]
 [ 4   4   1   1 391]]
Lemmatization
 [[495   3   8   1   3]
 [ 2 379   3   1   1]
 [ 8   3 401   3   2]
 [ 1   0   1 509   0]
 [ 4   3   1   2 391]]

Stem collisions in BBC vocabulary:
  organ     <- ['organic', 'organism', 'organizations', 'organizing', 'organs']
  market    <- ['market', 'marketed', 'marketing', 'markets']
  compani   <- ['companies', 'company']
  busi      <- ['business', 'businesses', 'busy']
  technolog <- ['technological', 'technologically', 'technologies', 'technology']
  commun    <- ['communicate', 'communicated', 'communicating', 'communication', 'communications', 'communicator', 'communism',
'communities']
  gener     <- ['general', 'generally', 'generate', 'generated', 'generates', 'generating', 'generation', 'generations']
  univers   <- ['universal', 'universe', 'universities', 'university']
  execut    <- ['executed', 'execution', 'executive', 'executives']
```

Question 5 – Identify the Error, Rectify It, and Analyse the Output

Original code

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("BBCNews.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

1. Error Identification

1. **Stemmer applied to a whole document.** apply(ps.stem) passes each full article string to stem(), which is defined for a single word.
1. **No tokenisation or normalisation.** Text is never split into words, lower-cased, or cleaned of punctuation, so even a corrected call would see tokens like news, or "Hello.
2. **Column name mismatch.** The dataset's column is text (lower-case); data["Text"] raises KeyError: 'Text'. The file name must also match the file actually downloaded.
3. **No missing-value guard.** A NaN cell would raise an error because stem() expects a string.

2. Root Cause: why the output is incorrect

Porter's algorithm treats its input as **one word** and works on its ending. Given a 4,333-character article, it returns almost the same string (4,333 characters back), with only the tail of the string touched, so no word inside the article is stemmed. The "Processed" column therefore looks unchanged, and downstream counts would still treat *markets, marketing, marketed* as different words.

3. Correction and Justification

- Select the right column and tokenise with a regular expression after lower-casing (re.findall(r"[a-z]+", ...)), which also removes punctuation and possessive fragments.
- Stem **each token** and join the result back: " ".join(ps.stem(t) for t in tokens).
- Convert values with str(text) to guard against NaN.

Corrected program, executed on the BBC News dataset, with output

```
"""AT3-Q5: Buggy vs corrected Porter stemming on BBC News"""
import re, collections, pandas as pd
from nltk.stem import PorterStemmer, WordNetLemmatizer
ps, lem = PorterStemmer(), WordNetLemmatizer()
data = pd.read_csv("../data/bbc.csv")                    # columns: category, text  (file has no "Text" column)

# ---- ORIGINAL (buggy) behaviour ----------------------------------------
print("ORIGINAL CODE")
```

```
try:
      = data["Text"]
except KeyError as e:
   print("  Error 1 -> KeyError:", e, "(real column is 'text', lower-case)")
whole = ps.stem(data["text"].iloc[0])                    # stem() on the full document string
print("  Error 2 -> ps.stem(whole document) returns ONE 'stem' for the whole text:")
print("   ", repr(whole[:90]), "...")
print("   input length:", len(data['text'].iloc[0]), "chars | output length:", len(whole), "chars (only the tail of the string is touched)\n")

# ---- CORRECTED ----------------------------------------------------------
def stem  text(text):
   tokens = re.findall(r"[a-z]+", str(text).lower())      # tokenise + normalise
   return " ".join(ps.stem(t) for t in tokens)            # stem word-by-word
data["Processed"] = data["text"].apply(stem  text)
print("CORRECTED OUTPUT")
print(data["Processed"].head(3).str[:110].to_string(), "\n")

# ---- Original vs stemmed words ------------------------------------------------
words = collections.Counter(w for t in data.text for w in re.findall(r"[a-z]+", t.lower()))
sample = ["organization", "running", "studies", "universities", "government", "generally", "operation", "companies", "technology",
"news"]
print(f"{'Original':<15}{'Stemmed':<12}")
for w in sample: print(f"{w:<15}{ps.stem(w):<12}")

# ---- >= 20 incorrect stemming cases -------------------------------------------------------
cases = ["universal", "university", "organization", "organic", "general", "generate", "operation", "operative",
         "police", "policy", "executive", "execute", "national", "nation", "communism", "community",
         "marketing", "market", "authority", "author", "political", "politics", "reply", "replied", "technology", "technical",
         "arsenal", "arsenic", "wander", "wand", "news", "new", "experience", "experiment", "infectious", "infection",
         "relativity", "relative", "medical", "medicine"]
rows = []
for w in cases:
   if w in words: rows.append((w, ps.stem(w), words[w]))
print("\nCandidate wrongly stemmed words found in BBC News:", len(rows))
for r in rows: print(f"  {r[0]:<14}->{r[1]:<10} (freq {r[2]})")
ORIGINAL CODE
  Error 1 -> KeyError: 'Text' (real column is 'text', lower-case)
  Error 2 -> ps.stem(whole document) returns ONE 'stem' for the whole text:
    'tv future in the hands of viewers with home theatre systems  plasma high-definition tvs  a' ...
   input length: 4333 chars | output length: 4333 chars (only the tail of the string is touched)

CORRECTED OUTPUT
0    tv futur in the hand of viewer with home theat...
1    worldcom boss left book alon former worldcom b...
2    tiger wari of farrel gambl leicest say they wi...

Original       Stemmed
organization   organ
running        run
studies        studi
universities   univers
government     govern
generally      gener
operation      oper
companies      compani
technology     technolog
news           news

Candidate wrongly stemmed words found in BBC News: 38
  universal     ->univers    (freq 35)
  university    ->univers    (freq 134)
  organization  ->organ      (freq 3)
  organic       ->organ      (freq 7)
  general       ->gener      (freq 366)
  generate      ->gener      (freq 22)
  operation     ->oper       (freq 65)
  operative     ->oper       (freq 2)
  police        ->polic      (freq 262)
  policy        ->polici     (freq 199)
  executive     ->execut     (freq 314)
  execute       ->execut     (freq 3)
  national      ->nation     (freq 316)
  nation        ->nation     (freq 68)
  communism     ->commun     (freq 2)
  community     ->commun     (freq 128)
  marketing     ->market     (freq 83)
```

```
market       ->market     (freq 702)
authority    ->author     (freq 43)
author       ->author     (freq 34)
political    ->polit      (freq 232)
politics     ->polit      (freq 106)
reply        ->repli      (freq 4)
replied      ->repli      (freq 22)
technology   ->technolog  (freq 561)
technical    ->technic    (freq 76)
arsenal      ->arsen      (freq 159)
arsenic      ->arsen      (freq 2)
wander       ->wander     (freq 1)
news         ->news       (freq 518)
new          ->new        (freq 1978)
experience   ->experi     (freq 121)
experiment   ->experi     (freq 10)
infectious   ->infecti    (freq 3)
infection    ->infect     (freq 13)
relative     ->rel        (freq 10)
medical      ->medic      (freq 46)
medicine     ->medicin    (freq 9)
```

4. Application of Concepts: analysis of 26 incorrect stemming cases

Original words versus stems from the BBC vocabulary. Each case was classified by the kind of error and by the morphology involved: **26 cases, of which 23 are derivational, 1 is mixed inflectional + derivational, and 2 are inflectional (orthographic y → i rule).**

#	Original	Stem	Freq.	Error type	Morphology	Explanation
1	universal	univers	35	Over-stemming	Derivational	merged with university, universe
2	university	univers	134	Over-stemming	Derivational	merged with universal, universe
3	universe	univers	11	Over-stemming	Derivational	different lexeme (cosmos)
4	organization	organ	3	Over-stemming	Derivational	merged with organ (body part)
5	organic	organ	7	Over-stemming	Derivational	merged with organ, organization
6	organizing	organ	1	Over-stemming	Inflectional + derivational	-ing is safe to drop, but -ize and -ation also lost
7	general	gener	366	Over-stemming	Derivational	merged with generate, generation
8	generate	gener	22	Over-stemming	Derivational	verb merged with adjective general
9	generally	gener	32	Over-stemming	Derivational	-ly adverb merged with general
10	operation	oper	65	Over-stemming	Derivational	surgery/business sense merged with operative
11	operative	oper	2	Over-stemming	Derivational	adjective/noun merged with operation
12	execute	execut	3	Over-stemming	Derivational	verb merged with executive
13	executive	execut	314	Over-stemming	Derivational	manager sense merged with execute
14	communism	commun	2	Over-stemming	Derivational	ideology merged with community
15	community	commun	128	Over-stemming	Derivational	merged with communism, communication

#	Original	Stem	Freq.	Error type	Morphology	Explanation
16	authority	author	43	Over-stemming	Derivational	merged with author
17	arsenal	arsen	159	Over-stemming	Derivational (spurious)	merged with arsenic; unrelated words
18	arsenic	arsen	2	Over-stemming	Derivational (spurious)	merged with arsenal; unrelated words
19	experience	experi	121	Over-stemming	Derivational	merged with experiment
20	experiment	experi	10	Over-stemming	Derivational	merged with experience
21	infectious	infecti	3	Under-stemming	Derivational	stem infecti differs from infection -> infect
22	medical	medic	46	Under-stemming	Derivational	medic vs medicine -> medicin: family split
23	technology	technolog	561	Under-stemming	Derivational	technolog vs technical -> technic: family split
24	reply	repli	4	Non-word stem	Inflectional (y->i rule)	repli is not a word
25	policy	polici	199	Non-word stem	Inflectional (y->i rule)	polici is not a word
26	relative	rel	10	Over-truncation	Derivational	rel is not a word; stem truncated too far

Interpretation. Almost all errors are **derivational**: Porter removes suffixes such as *-ation, -ive, -ity, -ize, -al, -ic, -ous* that build new lexemes, which either merges unrelated words (over-stemming: *arsenal / arsenic*), splits one family (under-stemming: *infectious / infection*), or leaves non-words (*repli, polici*). Only the true inflectional endings (*-s, -ed, -ing*) are safe to strip, and the y → i cases are an orthographic by-product of that stripping.

Question 7 – Error Analysis of Morphological Feature Extraction

Original code

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
stemmer = PorterStemmer()
documents = ["running runners runs", "studies studied studying", "organization organized organizer"]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)
features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]
print(features)
```

1. Error Identification

- **Stemming is applied after feature extraction.** The count matrix X is built from the *unstemmed* words; only the list of column names is stemmed afterwards.
- **Matrix and names are out of sync:** the columns of X still hold counts for 9 separate words, but the printed names show only 4 distinct stems, so the printed vocabulary is not what the model uses.

2. Root Cause Analysis

Stemming a list of column labels does not merge the columns. On the toy corpus the list becomes ['organ', 'organ', 'organ', 'runner', 'run', 'run', 'studi', 'studi', 'studi']: **9 entries, 4 unique, 5 duplicates.** The model therefore learns three separate weights for *organ*, *organized* and *organizer*, so the counts of one concept are split across columns: redundant vocabulary, more parameters, and diluted statistics. On the real corpus (below) the same mistake keeps **16,820 redundant columns**.

3. Correction and Justification

Stem **inside the analyzer**, so tokens are stemmed before they are counted and equal stems share a column:

```
def stem_analyzer(doc):
    return [stemmer.stem(w) for w in re.findall(r"(?u)\b[a-zA-Z]{2,}\b", doc.lower())]
vectorizer = CountVectorizer(analyzer=stem  analyzer)
X = vectorizer.fit_transform(documents)
```

Corrected toy output: features ['organ', 'run', 'runner', 'studi'], matrix shape (3 × 4) with counts [[0 2 1 0], [0 0 0 3], [3 0 0 0]]: the three study forms now fall in one column (count 3), and the three organization forms in one column (count 3). A cache (lru_cache) avoids re-stemming repeated words.

4. Application: execution on 20 Newsgroups

11,314 posts, 20 classes, 75/25 stratified split, English stop-words removed, Multinomial Naïve Bayes.

Pipeline	Features used by model	Distinct stems	Accuracy	Time
Original (stem after vectorising)	72,054	55,234	87.49 %	2.6 s
Corrected (stem before vectorising)	55,234	55,234	87.24 %	2.0 s

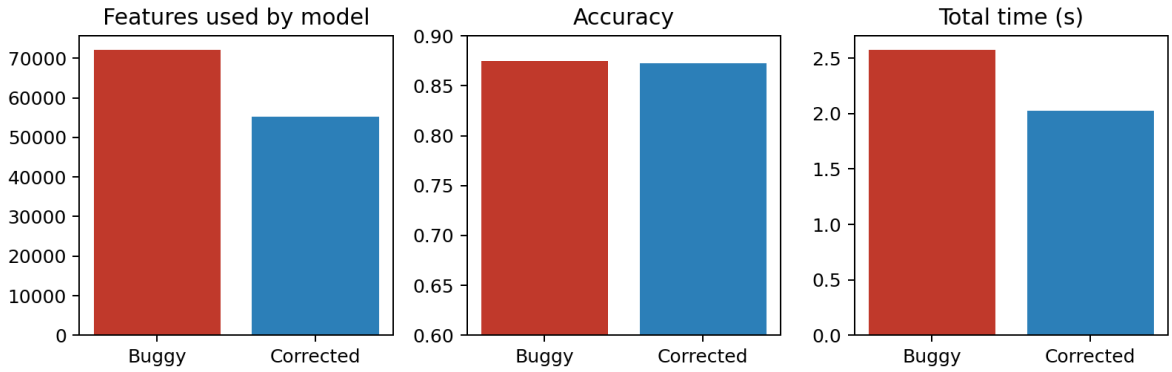


Figure 3. Original and corrected pipeline on 20 Newsgroups

Program and output

```
"""AT3-Q7: Stemming AFTER vectorisation (bug) vs BEFORE (fix) - 20 Newsgroups"""
import re, time, json, pandas as pd, numpy as np
from functools import lru_cache
from nltk.stem import PorterStemmer
from sklearn.feature  extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy  score

stemmer = PorterStemmer()

# ---------- 1. Toy example from the question ----------
documents = ["running runners runs", "studies studied studying", "organization organized organizer"]
vec = CountVectorizer(); X = vec.fit_transform(documents)
bug = [stemmer.stem(w) for w in vec.get  feature  names  out()]
print("BUGGY  feature list     :", bug)
print("       len =", len(bug), "| unique =", len(set(bug)), "-> duplicates:", len(bug) - len(set(bug)))
print("       matrix shape     :", X.shape, "(columns were NOT merged)")

@lru  cache(maxsize=None)
def stem  cached(w): return stemmer.stem(w)
def stem_analyzer(doc):
    return [stem  cached(w) for w in re.findall(r"(?u)\b[a-zA-Z]{2,}\b", doc.lower())]
vec2 = CountVectorizer(analyzer=stem  analyzer); X2 = vec2.fit  transform(documents)
print("FIXED  feature list     :", list(vec2.get_feature_names_out()))
print("       matrix shape     :", X2.shape, "\n", X2.toarray(), "\n")

# ---------- 2. Real corpus: 20 Newsgroups ----------
df = pd.read_json("../data/ng.json").dropna(subset=["content"])
```

```
df = df[df.content.str.len() > 50]
text = df.content.str.replace(r"^.*?\n\n", "", regex=True, n=1)         # strip e-mail headers
y = df.target
print("Documents:", len(df), "| classes:", y.nunique())
Xtr, Xte, ytr, yte = train  test  split(text, y, test  size=0.25, stratify=y, random  state=42)

def run(analyzer, label, post_stem=False):
    t0 = time.time()
    v = CountVectorizer(analyzer=analyzer, min  df=1) if analyzer != "word" else CountVectorizer(stop  words="english")
    A = v.fit_transform(Xtr); B = v.transform(Xte)
    names = v.get  feature  names  out()
    if post_stem:
        names_stemmed = [stem_cached(w) for w in names]                  # buggy: stem the vocabulary list only
        size  after = len(set(names  stemmed)); listed = len(names  stemmed)
    else:
        size_after = listed = len(names)
    clf = MultinomialNB(alpha=0.1).fit(A, ytr)
    acc = accuracy  score(yte, clf.predict(B)); sec = time.time() - t0
    return dict(label=label, listed=listed, distinct=size_after, acc=acc, time=sec)

import re as  re
STOP = set(__import__("sklearn.feature_extraction.text", fromlist=["ENGLISH_STOP_WORDS"]).ENGLISH_STOP_WORDS)
def plain  analyzer(doc): return [w for w in  re.findall(r"(?u)\b[a-zA-Z]{2,}\b", doc.lower()) if w not in STOP]
def stem_analyzer2(doc): return [stem_cached(w) for w in plain_analyzer(doc)]

res = [run(plain_analyzer, "Original (stem after vectorizing - buggy)", post_stem=True),
       run(stem_analyzer2, "Corrected (stem before vectorizing)")]
print(f"\n{'Pipeline':<46}{'features listed':>16}{'distinct':>10}{'accuracy':>10}{'time(s)':>9}")
for r in res: print(f"{r['label']:<46}{r['listed']:>16}{r['distinct']:>10}{r['acc']:>10.4f}{r['time']:>9.1f}")
json.dump(res, open("q7.json", "w"))

import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 3, figsize=(9, 3))
lab = ["Buggy", "Corrected"]; cols = ["#c0392b", "#2c7fb8"]
ax[0].bar(lab, [res[0]["listed"], res[1]["listed"]], color=cols); ax[0].set_title("Features used by model")
ax[1].bar(lab, [res[0]["acc"], res[1]["acc"]], color=cols); ax[1].set  ylim(0.6, 0.9); ax[1].set  title("Accuracy")
ax[2].bar(lab, [res[0]["time"], res[1]["time"]], color=cols); ax[2].set  title("Total time (s)")
plt.tight_layout(); plt.savefig("q7_chart.png", dpi=180)
BUGGY  feature list      : ['organ', 'organ', 'organ', 'runner', 'run', 'run', 'studi', 'studi', 'studi']
       len = 9 | unique = 4 -> duplicates: 5
       matrix shape      : (3, 9) (columns were NOT merged)
FIXED  feature list      : ['organ', 'run', 'runner', 'studi']
       matrix shape      : (3, 4)
 [[0 2 1 0]
 [0 0 0 3]
 [3 0 0 0]]

Documents: 11314 | classes: 20

Pipeline                                 features listed  distinct  accuracy  time(s)
Original (stem after vectorizing - buggy)              72054     55234    0.8749      2.6
Corrected (stem before vectorizing)                    55234     55234    0.8724      2.0
```

5. Interpretation

- **Vocabulary:** 72,054 → 55,234 features (−23.3 %). The model now has one parameter per stem instead of several for the same stem.
- **Accuracy:** essentially unchanged (87.49 % → 87.24 %, a 0.25-point difference on one split of 2,829 test posts, which is within normal noise). The correction is therefore a **correctness and efficiency fix, not an accuracy boost** in this experiment; Naïve Bayes copes well with redundant columns.
- **Time:** the corrected run took 2.0 s versus 2.6 s in this single timing, because it builds a smaller matrix; timings are indicative only.
- **Why it still matters:** a vocabulary that actually matches the stems gives smaller memory use, better pooling of rare word forms, more stable weights for linear models, and a feature list that can be inspected and debugged.

Summary of Findings

Q	Main error	Fix	Key result
1	Stemming before morphological analysis; infectious → infecti	POS-aware lemmatisation + curated derivational family	Recall for “infectious” 5.4 % with Porter, versus 95 % for the other forms
3	Stems merge different lexemes; possessive handled badly	Lemmatisation; clean tokens	No accuracy difference (97.8 % vs 97.8 %); vocabulary −32 % (Porter), −20 % (lemma)
5	stem() applied to whole text; wrong column	Tokenise, then stem per word	26 stemming errors analysed; 23 derivational, 1 mixed, 2 inflectional
7	Stemming after CountVectorizer	Stem inside the analyzer	Vocabulary −23.3 %; accuracy unchanged